

GUÍA GESTIÓN DE USUARIOS

TABLA DE CONTENIDO

Guía Gestión de Usuarios	1	
Gestión de usuarios API	2	
Llave Secreta	2	
Encriptar contraseñas	2	
Middleware		2
Endpoints disponibles	2	
Gestión de usuarios Angular	3	
Servicio de autenticación	3	
Tabla resumen de variables, Signals y propiedades calculadas		3
Tabla resumen de métodos del servicio		4
shareReplay()		5
Interceptor	5	
Clonado de la petición		5
app.config.ts	6	
Inicialización automática de la sesión		6
Guard 6		
Guard de autenticación		6
Guard por rol		7
Uso de la información del usuario autenticado	7	
Mostrar la información en la plantilla		8
Restringir el acceso a una ruta por autenticación	8	
Configuración de una ruta protegida		8
Restringir el acceso a una ruta según el rol	9	
Ruta permitida para varios roles		9
Restringir visualmente elementos según el rol	9	
Validar varios roles con tieneRol()		10

GESTION DE USUARIOS API

LLAVE SECRETA

Para encriptar es necesario que establezca los siguientes aspectos:

- Establezca en el archivo .env la variable SECRET KEY

```
JWT_SECRET="vj_utn_2026"
```

Sí desea puede generar una nueva llave secreta ejecutan el siguiente comando

```
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

ENCRIPtar CONTRASEÑAS

Las contraseñas de los usuarios registrados deben estar **encriptadas** para poder realizar la comparación.

Ejecute los seeder nuevamente para que los usuarios cuenten con contraseñas correctas

MIDDLEWARE

Copie authMiddleware.ts en la carpeta middleware

Protege las rutas privadas del API. Su función es verificar que el cliente envíe un **token JWT válido** en el encabezado Authorization antes de permitir el acceso al recurso solicitado.

Funcionamiento

- Obtiene el encabezado Authorization de la petición.
- Verifica que el formato sea Bearer <token>.
- Valida el token utilizando la clave secreta (JWT_SECRET).
- Extrae la información del usuario (id, email y role) almacenada dentro del token.
- Guarda esos datos en request.user para que puedan utilizarse en los controladores y servicios.
- Si el token no existe, tiene un formato incorrecto, es inválido o expiró, responde con **401 Unauthorized** y no permite continuar la ejecución de la ruta.

ENDPOINTS DISPONIBLES

Método	Ruta relativa	Protección	Uso en Angular	Descripción técnica
GET	/	No aplicada en la ruta mostrada	Listado de usuarios	Consulta todos los usuarios almacenados en la base de datos.
POST	/register	Validación DTO	Registro	Valida los datos recibidos, verifica que el correo no exista, cifra la contraseña con bcrypt, crea el usuario en la base de datos y devuelve la información del usuario sin incluir la contraseña.
POST	/login	Validación DTO	Inicio de sesión	Valida el correo y la contraseña, compara la contraseña cifrada utilizando bcrypt, genera un JWT con el identificador, correo y rol del usuario, y

				devuelve el token para autenticación de las siguientes peticiones.
GET	/perfil	Bearer Token	Restaurar sesión y mostrar perfil	Utiliza el middleware de autenticación para validar el JWT, extrae el usuario autenticado desde request.user, consulta nuevamente la información en la base de datos y devuelve el perfil sin exponer la contraseña.
GET	/:id	No aplicada en la ruta mostrada	Detalle de usuario	Obtiene el identificador recibido en la URL, busca el usuario correspondiente en la base de datos y devuelve su información. Si el usuario no existe, responde con 404 Not Found.

GESTIÓN DE USUARIOS ANGULAR

SERVICIO DE AUTENTICACION

Copie y pegue dentro de la carpeta share el servicio **AuthenticationService**.

Este servicio centraliza toda la lógica relacionada con la autenticación de usuarios. Es el encargado de comunicarse con el API, almacenar el estado de la sesión y proporcionar la información del usuario autenticado al resto de la aplicación.

TABLA RESUMEN DE VARIABLES, SIGNALS Y PROPIEDADES CALCULADAS

Elemento	Tipo	Descripción técnica
tokenKey	string	Nombre de la clave utilizada para guardar y recuperar el JWT desde localStorage.
_token	Signal<string null>	Almacena internamente el token JWT actual. Su valor inicial se recupera desde localStorage.
_usuario	Signal<Usuario null>	Almacena internamente la información del usuario autenticado obtenida desde /usuario/perfil.
_cargandoSesion	Signal<boolean>	Indica si se está ejecutando una operación de inicio o restauración de sesión.
_sesionIniciada	Signal<boolean>	Indica si Angular ya terminó de comprobar si existe una sesión almacenada.
solicitudPerfilActual	Observable<Usuario null> null	Guarda temporalmente la solicitud activa a /perfil para evitar peticiones duplicadas.
token	Signal de solo lectura	Expone _token sin permitir que los componentes modifiquen directamente su valor.

usuario	Signal de solo lectura	Expone la información del usuario autenticado a componentes, Guards e interceptores.
cargandoSesion	Signal de solo lectura	Permite mostrar indicadores de carga mientras se procesa la autenticación.
sesionIniciada	Signal de solo lectura	Permite saber si ya terminó la restauración inicial de la sesión.
autenticado	computed<boolean>	Devuelve true cuando existe un token y el perfil del usuario fue recuperado correctamente.
rol	computed<Role null>	Obtiene el rol del usuario autenticado. Devuelve null cuando no existe sesión.
esAdmin	computed<boolean>	Comprueba si el rol actual corresponde a Role.ADMIN.

TABLA RESUMEN DE MÉTODOS DEL SERVICIO

Método	Visibilidad	Retorno	Descripción técnica
login(credenciales)	Público	Observable<Usuario>	Envía las credenciales al API, guarda el JWT, consulta el perfil y almacena el usuario autenticado.
obtenerPerfil()	Público	Observable<Usuario>	Consume GET /usuario/perfil y devuelve la información del usuario asociada con el token.
inicializarSesion()	Público	Observable<Usuario null>	Comprueba el token almacenado al iniciar Angular y restaura la sesión cuando corresponde.
cargarPerfil()	Público	Observable<Usuario null>	Recupera el perfil usando el token existente y evita solicitudes simultáneas mediante shareReplay().
registrar(datos)	Público	Observable<Usuario>	Envía los datos del nuevo usuario al endpoint de registro y devuelve el usuario creado.
logout(redirigir)	Público	void	Elimina el token, limpia el usuario y redirecciona al Login cuando redirigir es true.
tieneRol(rolesPermitidos)	Público	boolean	Comprueba si el rol del usuario actual se encuentra dentro del arreglo de roles permitidos.

obtenerToken()	Público	string null	Devuelve el JWT actual. Es utilizado principalmente por el interceptor de autenticación.
guardarToken(token)	Privado	void	Valida, limpia y almacena el token tanto en localStorage como en el Signal _token.
leerTokenAlmacena do()	Privado	string null	Recupera el token desde localStorage y descarta valores vacíos o inválidos.
limpiarSesion()	Privado	void	Elimina todo el estado relacionado con la autenticación y cancela la referencia a la solicitud de perfil.
obtenerErrorAutenticacion(error)	Privado	Error	Convierte errores HTTP en mensajes claros que puedan mostrarse en la interfaz.

SHAREREPLAY()

Permite compartir el resultado de una única petición HTTP entre varios suscriptores.

Si varios componentes o Guards solicitan el perfil al mismo tiempo, todos reciben la misma respuesta sin generar nuevas solicitudes al API.

En este caso:

- **bufferSize: 1** conserva la última respuesta obtenida (el usuario).
- **refCount: false** mantiene la solicitud compartida hasta que finaliza, aunque cambie el número de suscriptores.

Evita que, al iniciar la aplicación, el authGuard, roleGuard y otros componentes ejecuten simultáneamente varias peticiones a **GET /usuario/perfil**. En su lugar, todos reutilizan la misma solicitud y la misma respuesta.

INTERCEPTOR

Copiar **auth.interceptor.ts** en `src/app/core/interceptors`

Este interceptor agrega automáticamente el token JWT a todas las peticiones protegidas que realiza Angular hacia el API.

- Antes de enviar una petición, el interceptor consulta el token almacenado en AuthService.
- Evita que cada servicio tenga que agregar manualmente el encabezado de autenticación.

Copiar y reemplazar **src/app/core/interceptors/http-error.interceptor.ts**

CLONADO DE LA PETICIÓN

Las peticiones HTTP son inmutables.

Por esta razón se crea una copia utilizando: `request.clone()`

En dicha copia se agrega: `Authorization: Bearer TOKEN`

APP.CONFIG.TS

Configura los proveedores principales de la aplicación Angular.

En el archivo app.config.ts se deben registrar dos interceptores:

- **authInterceptor**
 - Su responsabilidad es incluir automáticamente el token JWT en las peticiones autenticadas.
- **httpErrorInterceptor (actualizado)**
 - Centraliza el manejo de errores provenientes del servidor.

INICIALIZACIÓN AUTOMÁTICA DE LA SESIÓN

Se utiliza: `provideAppInitializer()`

Antes de que Angular termine de cargar:

1. Verifica si existe un token;
2. Consulta el perfil del usuario;
3. Restaura la sesión automáticamente.

Este proceso evita que el usuario tenga que volver a iniciar sesión al actualizar la página.

GUARD

Copiar en `src/app/core/guards`

- `auth.guard.ts`
- `role.guard.ts`

GUARD DE AUTENTICACIÓN

Protege las rutas que únicamente pueden ser visitadas por usuarios autenticados.

VALIDACIÓN DE AUTENTICACIÓN

Antes de ingresar a una ruta protegida, el Guard verifica si existe un usuario autenticado.

Si la sesión continúa activa, permite navegar normalmente.

RESTAURACIÓN DE SESIÓN

Si la aplicación acaba de iniciar y todavía no conoce el usuario autenticado, solicita nuevamente el perfil utilizando el token almacenado.

Esto evita que un usuario autenticado sea redireccionado al Login simplemente por actualizar la página.

REDIRECCIÓN

Si no existe una sesión válida, el Guard redirecciona automáticamente a: `/login`

GUARD POR ROL

Controla el acceso a las rutas según el rol del usuario autenticado.

LECTURA DE ROLES PERMITIDOS

Obtiene desde la configuración de la ruta el arreglo de roles autorizados.

Ejemplo: roles: [Role.ADMIN]

COMPARACIÓN DE ROLES

Obtiene el rol del usuario autenticado y verifica si pertenece al conjunto de roles permitidos.

La validación se realiza utilizando el método tieneRol() del AuthService, centralizando la lógica de autorización.

REDIRECCIÓN POR PERMISOS

Si el usuario está autenticado pero no posee el rol requerido, el Guard redirecciona automáticamente a:

/sin-autorizacion

USO DE LA INFORMACIÓN DEL USUARIO AUTENTICADO

Una vez que el usuario inicia sesión, AuthService guarda su información en el Signal usuario. Este estado puede consultarse desde cualquier componente que necesite mostrar datos personales, validar permisos o adaptar la interfaz

Obtener la instancia global del servicio de autenticación:

```
private readonly authService = inject(AuthService);
```

Signal de solo lectura que contiene el usuario autenticado:

```
readonly usuario = this.authService.usuario
```

MOSTRAR LA INFORMACIÓN EN LA PLANTILLA

```
@if (usuario(); as usuarioActual) {  
  <div class="user-menu-info">  
    <strong>  
      {{ usuarioActual.nombre }}  
    </strong>  
    <span>  
      {{ usuarioActual.email }}  
    </span>  
    <small>  
      {{ nombreRol() }}  
    </small>  
  </div>  
  <mat-divider></mat-divider>  
}
```

RESTRINGIR EL ACCESO A UNA RUTA POR AUTENTICACIÓN

Las rutas privadas deben protegerse con authGuard. Este Guard comprueba que exista una sesión válida antes de permitir la navegación

CONFIGURACIÓN DE UNA RUTA PROTEGIDA

```
{  
  path: 'perfil',  
  component: Perfil,  
  title: 'Mi perfil',  
  canActivate: [authGuard],  
},
```

Indica que Angular debe ejecutar el Guard antes de crear el componente asociado a la ruta.

El acceso se permite cuando:

- Existe un token;
- El perfil fue recuperado correctamente;
- AuthService.autenticado() devuelve true.

Si no existe una sesión válida, el usuario es redireccionado a: /login

RESTRINGIR EL ACCESO A UNA RUTA SEGÚN EL ROL

Cuando una ruta requiere un rol específico, se agregan `authGuard`, `roleGuard` y la propiedad `data.roles`.

```
{
  path: 'admin/ordenes',
  component: OrdenesList,
  title: 'Gestión de órdenes',
  canActivate: [authGuard, roleGuard],
  data: { roles: [Role.ADMIN] },
},
```

Angular ejecuta los Guards en orden.

Primero comprueba que el usuario esté autenticado. Después valida que tenga uno de los roles permitidos.

Define los roles autorizados para ingresar a la ruta.

El `roleGuard` lee esta información y la compara con el rol almacenado en `AuthService`.

RUTA PERMITIDA PARA VARIOS ROLES

```
{
  path: 'admin/ordenes',
  component: OrdenesList,
  title: 'Gestión de órdenes',
  canActivate: [authGuard, roleGuard],
  data: { roles: [Role.ADMIN, Role.USER] },
},
```

El usuario puede ingresar cuando su rol coincide con cualquiera de los valores definidos en el arreglo.

RESTRINGIR VISUALMENTE ELEMENTOS SEGÚN EL ROL

La restricción visual consiste en ocultar botones, enlaces o secciones que el usuario no debe utilizar.

Esta validación mejora la experiencia de usuario, pero no reemplaza la protección de rutas.

AuthService ya expone una propiedad calculada llamada **esAdmin**

```
private readonly authService = inject(AuthService)
readonly esAdmin = this.authService.esAdmin
```

En la plantilla:

```
@if (esAdmin()) {
<button type="button">
  Crear usuario
</button>
}
```

El botón solamente se crea cuando el rol actual es ADMIN.

Si el usuario cambia o cierra sesión, Angular actualiza automáticamente la interfaz.

VALIDAR VARIOS ROLES CON TIENEROL()

```
/*  
 * Determina si un elemento del menú puede mostrarse.  
 * Sin roles:  
 * el elemento es público.  
 * Con roles:  
 * AuthService comprueba si el usuario posee  
 * uno de los roles permitidos.  
 */  
puedeMostrar(item: MenuItem): boolean {  
    if (!item.roles?.length) {  
        return true  
    }  
    return this.authService.tieneRol(  
        item.roles  
    )  
}
```

tieneRol() recibe un arreglo de roles permitidos y devuelve true cuando el rol actual pertenece a ese arreglo.